Performance Evaluation of Navigation Schemes for Autonomous Exploration of a Secondary Explorer Rover in a Martian Lava Cave

Antar Mazumder
Dept. of MTE, RUET
Rajshahi 6204, Bangladesh
antar.mte@gmail.com

Md. A. K. Niloy
Dept. of MTE, RUET
Rajshahi 6204, Bangladesh
niloyglhs123@gmail.com

Md. Fahad Sahed
Dept. of MTE, RUET
Rajshahi 6204, Bangladesh
sahed.fahad@gmail.com

Zarin Anjum Madhiha
Dept. of EEE, RUET
Rajshahi 6204, Bangladesh
madhihazarinanjum@gmail.com

Faisal R. Badal
Dept. of MTE, RUET
Rajshahi 6204, Bangladesh
faisalrahman1312@gmail.com

Jalal Uddin
Dept. of MTE, RUET
Rajshahi 6204, Bangladesh
com.jalal.uddin@gmail.com

Md. Zahid Hossain
Dept. of MTE, RUET
Rajshahi 6204, Bangladesh
1608024@student.ruet.ac.bd

***Abstract*—Habitation in mars is one of the most ambitious goals of the 21st century as our permanent habitat, the earth is slowly losing its habitation qualities due to different disasters, pollution and rise of temperature. But there include several major challenges before we can make mars a habitable planet. Radiation, dusty storms, meteorites etc. makes the Martian surface one of the difficult places to live. The caves in the mars can be a viable solution to this problem as they can give protection against these threats. Lava caves are of particular interest in these case as these caves is similar to the lava caves found on earth. This research presents the design and performance evaluation of an autonomous secondary explorer rover in lava tubes of Mars. 3D & 2D mapping was done in a simulated environment and the performance of various path planning algorithms was evaluated in the lava cave. The experiment results show global path planners, especially A* can perform well if the entrance and exit are well defined while local path planners need human interventions several times even though they can avoid sudden dangers or landscape change inside the cave.**

***Index Terms*—SLAM, Mars Rover, Navigation, Path Planning, ROS, Gazebo.**

I. Introduction

Space exploration has reached a new high with the introduction of Ingenuity, also known as the quad-bladed "Mars Helicopter." Aside from being a simple flight test on a different planet, it has also established a clear indication that the addition of smaller secondary UAVs and UGVs with the primary rover can greatly improve the efficiency of the exploration procedure [1]. One excellent example would be the case where the rover's mission is to explore and collect data from a Martian lava cave.

Although the reduced gravity results in a larger diameter in Martian lava cave structures, such as lava tubes, the internal rocky obstacles and narrow sub-branches caused by volcanic processes [1] make manoeuvring the primary substantially large rover a difficult challenge. As a solution to this problem, this study proposes a small-scale UGV or rover that has more freedom of movement in such environments.

However, as we edge closer to the term "small-scale," new challenges emerge such as low-component carrying capability and weight optimization as seen in the case of ingenuity [2]. As a consequence, a small scale rover is more likely to have less power and component carrying capacity, necessitating optimization of allotted power and footprint consumption.As a solution, we propose a small-scale 4-wheeler UGV with a modular 360-degree rotating 2D LiDAR, wheel encoders, additional camera and IMU sensors such that it would consume less power and require fewer components for navigation procedure. Assuming that assertion, this study aims to evaluate various path planning algorithms, including A*, Dijkstra, and Rapidly-exploring Random Tree (RRT), for autonomous navigation inside a simulated Martian lava cave.

In the following pages, the paper is organised into five sections. The succeeding section walks us through a relevant background study followed by the proposed architecture in section III. Section IV demonstrates the results and provides a detailed corresponding discussion. Finally, the study meets its conclusion in section V.

II. Background Study

The capability of navigation is one of the most vital roles for any mobile robot. One of the most essentials is to avoid harmful circumstances. Robots have a capability that allows them to investigate particular spots in their surroundings.

Robot navigation can be pertained to a robot's capacity to establish its orientation in its conceptual framework and then design a course to some desired place [3]. To navigate in its region, the mobile robot or any other mobility device requires representation, a map of the world and the capacity to analyze that interpretation. Mobile robots develop increased intelligence as a result of machine learning algorithms, which

978-1-6654-6658-5/22/$31.00 IEEE

©2022 IEEE

boosts productivity [4]. The primary concern of the mobile robotics field is navigation systems and path planning. According to [5], mobile robots now successfully execute daily activities in a variety of sectors and can quickly manoeuvre in challenging surroundings. Without a doubt, path planning plays an essential part which is one of the most investigated issues in robotics for mobile robot navigation [6]. The path planning challenge for autonomous cars has been one of the most explored issues in the last decade. Consequently, sample-based algorithms for path planning have become increasingly relevant in the area of research. For mobile robot path planning, many algorithms are used. They are divided into two sections: classical methods and heuristic methods [7]. For the emerging robotics sector, researchers have recently concentrated on heuristic approaches including the Dijkstra algorithm and the A* algorithm. For path planning strategies, there is a traditional method RRT (Rapidly exploring Random Tree) that is employed. Currently, this is a cause of discomfort for researchers due to its asymptotic optimally. Dijkstra's Method is an algorithm that is used to identify the smallest distance, between an initial point to a destination node in a congested graph [8]. Dijkstra's variant algorithm is denoted by A* [9]. For determining the shortest path, the A* method is perhaps the most appropriate. According to studies A* is faster than Dijkstra. But using the heuristic function to find a better approach the primary distinction between A * and Dijkstra. One of the several path planning systems is RRT. The ultimate focus of RRT is to find locations predicated on the development of a random tree. The author explains [10] that in non-complex situations, RRT is smoother, it is determined that RRT methods are substantially quicker than A*, although the pathways identified by RRT are longer than A*. the author explains in comparison to the RRT method, the A* algorithm creates quicker pathways. The A* algorithm just investigates the quantities necessary for path creation, whereas the RRT investigate the entire volume.

III. System Architecture

This section walks us through a detailed overview of the system architecture adopted in this study. The hierarchy of the system phases followed in this study can be summarised from the figure 3.

For this study, it is not feasible to develop a real robot as the target environment is a Martian lava cave. Thus, we intend to utilize a simulated robot design in a simulated environment in Gazebo, a state of the art physics simulator that decently mimics the conditions of that of a Martian lava cave with dynamic gravity reduction and other functionalities.

A. Robot Design

The small-scale rover, Scout was designed in Fusion 360, an esteemed CAD software. The designed rover is a robot with dimensions $65 * 47 * 30$ cm.

It is a four-wheeled UGV that utilizes a skid-steer driving mechanism for adequate traction along the lava cave path. In this mechanism, the movement of the robot is determined by



Fig. 1. Robot Design

the relative movement between the left and right treads that can be summarised from the equation 1 [11] -

$$\begin{pmatrix} v_x \\ v_y \\ \omega_z \end{pmatrix} = fd \begin{pmatrix} V_L \\ V_R \end{pmatrix} \tag{1}$$

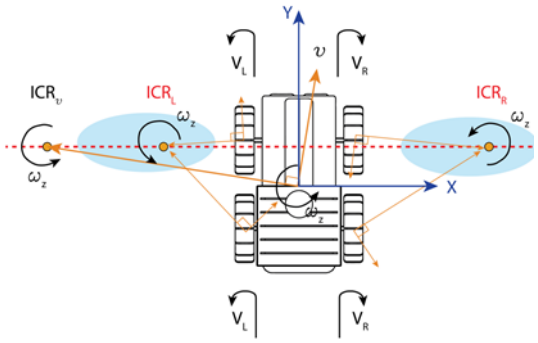


Fig. 2. Skid-Steer Mechanism of Scout

B. Software Integration

1) URDF Generation: URDF stands for Unified Robotic Description Format, which is, in short, the digital representation of the visual, functional and geometrical properties of the robot typically written in XML [12]. For us to simulate the rover design in Gazebo, first the CAD design needed to be transformed into its URDF, which had been done using a third-party and open-source plugin known as fusion2urdf, which allowed it to be imported and simulated in the Gazebo environment.

2) ROS: ROS stands for Robot Operating System. It is a reputed open-source meta-operating system dedicated to developing efficient robot software. Throughout the process, various features of ROS Noetic, such as nodes, topics, and messages had been utilized for interconnecting the process functionalities and data transmission between the nodes and the sensors. The meta-OS runs atop an Ubuntu 20.04 machine for additional functionalities.

C. Simulation Environment Setup

This procedure is carried out within the Gazebo simulator. We begin by constructing the cave structure, which is an open-source model provided by OpenRobotics, as follows:

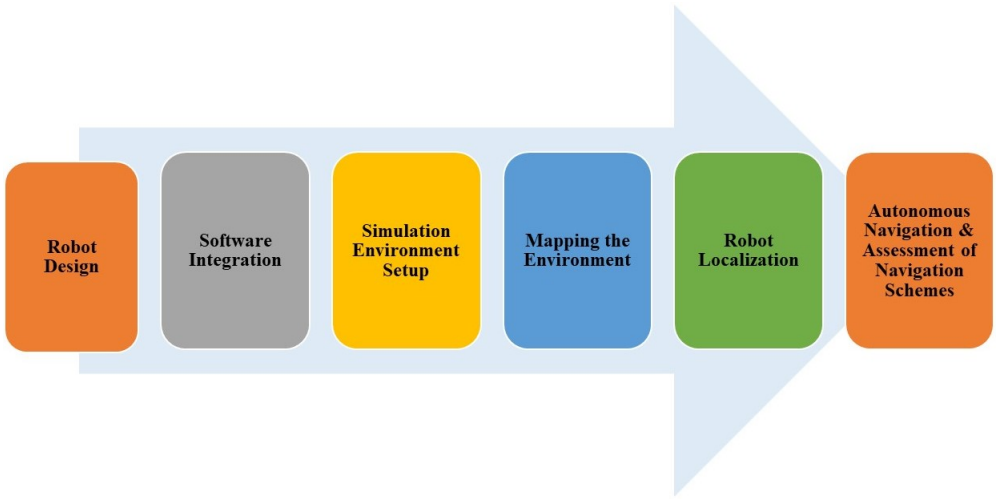


Fig. 3. System overview

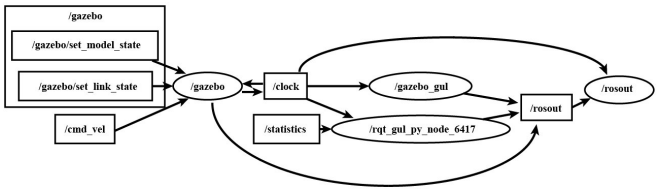


Fig. 4. A basic Interpretation of Interconnection of nodes and functionalities in ROS

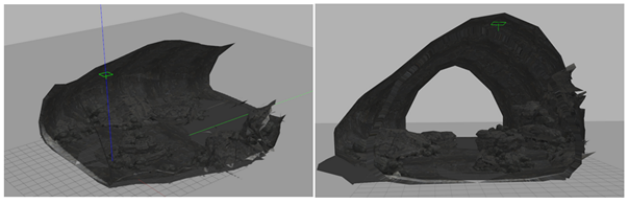


Fig. 5. Lava Cave Setup inside Gazebo

Next, we adjust the gravity of the environment using the rqt_reconfigure feature, which allows us to do so dynamically. We first set the value equal to the magnitude of Earth's gravitational acceleration and then change it to that of Mars.

D. Mapping the Environment

Mapping is one of the most critical phases of the system processes. It is crucial for robot localization and corresponding navigation scheme(s). In this study, we adopted a 2D mapping method- Gmapping and a 3D mapping one- OctoMap to see the difference as well as assess the trade-off we are to encounter by utilizing a 2D rotating LiDAR instead of a 3D one.

1) Gmapping: It is a particle filter-based approach that builds a map of surroundings from laser data. In our case, the laser data is received from the 2D LiDAR. In this approach, individual particles contain a map of the surroundings and together they estimate a model-based true-probability of the map and the pose of the robot relevant to the generated map [13].

2) OctoMap: OctoMap is one of the most advanced 3D mapping technologies used in robotic navigation. Unlike another popular algorithm, RTAB-Map, Octomap provides a more visually insightful 3D map that consumes significantly less memory space and can be saved as a.bt or. ot file and later visualized using RViz, Octovis, and other supporting visualization tools.

It stores map data received by the 3D lidar sensor used in our rover simulation in Octrees, which combine cells with similar occupied patterns and save the information in the parent cube [14]

E. Robot Localization

The ROS navigation stack's default localization method is AMCL or Adaptive Monte Carlo Localization. It is a particle-based localization method in which particles are dispersed across the generated map. When the robot iterates over previously explored map regions, particles accumulate at specific locations, allowing the robot's position relative to the map to be predicted. As a result, the robot is localized.

F. Autonomous Navigation & Assessment of Navigation Schemes

The autonomous navigation of the robot heavily depends on the navigation schemes that are the local and global path planning approaches. The default local planner offered by ROS is DWA or Dynamic Window Approach, which is utilized in the cases for A* and Dijkstra global path planners. The RRT algorithm picks up a different approach known as estimating local frontiers.

No comparison will be made among the local planners in this study, as the corresponding methods are out of the intended scope and also too diverse to make an assessment. However, the global path planners, namely A*, Dijkstra, and a

special local one, Rapidly-exploring Random Tree (RRT) will be the centre topics for performance bench-marking.

1) RRT Algorithm (Rapidly-Exploring Random Tree): Path exploration in high-dimensional space is significantly more difficult than in low-dimensional space. The RRT algorithm's core concept is to explore in the shape of a tree from the manipulator's beginning and to sample at random throughout feasible space in order to extend the branches and leaves of the exploring tree until it reaches the goal region [15].

A rapidly exploring random tree (RRT) is a space-filling tree that is randomly generated to effectively search non-convex, high-dimensional environments. It was introduced in [16] as an efficient data structure and sampling scheme to quickly search high-dimensional spaces that have both algebraic constraints (arising from obstacles) and differential constraints (arising from nonholonomy and dynamics).

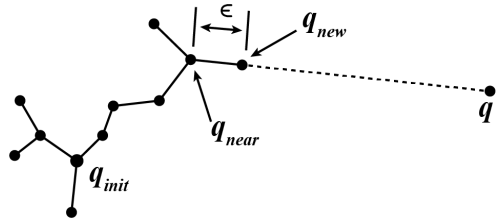


Fig. 6. The operation of RRT algorithm

2) A Search Algorithm:* A-Star algorithm, which combines the benefits of Best First Search and Dijkstra algorithm, can not only guarantee the discovery of the optimal path in a static environment but also enhance efficiency through heuristic search, making it suited for accurate demand planning [17]. This algorithm finds the next node in a 2D grid map based on the final function.

$$f(n) = g(n) + h(n) \tag{2}$$

Where the movement cost path from the beginning point to the node n is denoted by g(n); Due to the path uncertainty produced by unknown obstructions, h(n) refers to the path cost of travelling from the present node to the destination, which is also known as a heuristic function [10].

Manhattan distance and Euclidean distance are included in the distance computation of the heuristic function of the A-STAR algorithm:

$$h(n) = |x_n - x_g| + |y_g - y_n| \tag{3}$$

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2} \tag{4}$$

Where, the coordinates of target point G are (x_g, y_g) and the coordinates of the current node are (x_n, y_n). The Manhattan distance is represented by equation 3, and the Euclidean distance is represented by equation 4.

3) Dijkstra Algorithm: The Dijkstra algorithm's initial work is limited to finding the shortest route between two points. These points must be mathematically described by nodes in the graph topology.

The Dijkstra algorithm follows a straightforward approach. The traditional Dijkstra algorithm seeks to identify the shortest path between two nodes on a digraph $D = (N, W)$, where N denotes the set of all nodes and W denotes the set of connected nodes' weighted edges. The Dijkstra algorithm divides N into two groups. One is N_e and another one is N_u. The end nodes of the determined shortest path to source node N_s are grouped together as N_e.

The collection of nodes from N_u to N_s with the unknown shortest path is called N_u. In the first step, N_e contains only N_s. The nodes in Nu will be relocated to N_e in ascending order of the source node's shortest path length N_s until the set Nu is empty. The shortest path from N_s to $N_t i$ is the path that sequentially connects the source node N_s to all edges of any node $N_t i$. The length of the Dijkstra algorithm's shortest path is equal to the sum of the corresponding weights. As a result, the shortest path between the source node N_s and the target node N_t can be determined [10].

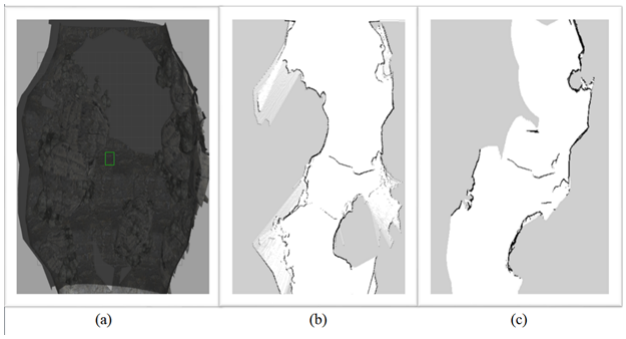


Fig. 7. (a) The simulated lava cave, (b) Map generated during manual Gmapping, (c) Map generated during RRT exploration

IV. Results and Discussion

As the name suggests, this section will elaborate on and discuss the results obtained from the simulation study. First, it will show the trade-off between a 2D and a 3D map of the same lava cave structure. Furthermore, it will address the research inquiries related to the navigation schemes.

A. Mapping

1) 2D Mapping: The 2D map of the lava cave structure was generated utilizing Gmapping. First, the general manual Gmapping was performed in the cases of A* and Dijkstra path planning, which is depicted in Fig. 7 (b). In the case of RRT exploration, the Gmapping process takes place simultaneously and the generated map is presented in Fig. 7 (c).

2) 3D Mapping: Fig. 8(b) below shows the 3D OctoMap of the simulated lava cave presented in Fig. 8(a). The corresponding 2D map is depicted in Fig. 8(c) to highlight the differences.

TABLE I
COMPARATIVE ANALYSIS BETWEEN MANUAL GMAPPING AND RRT GMAPPING

Parameters	2D Mapping	
	Manual Gmapping	RRT Gmapping
Method	Manual explorer	Autonomous explorer
Speed	Faster	Slower
Details	Finer	Often incomplete (3/5 Trials)
Sensor	Lidar + Wheel Odometry	Lidar + Wheel Odometry

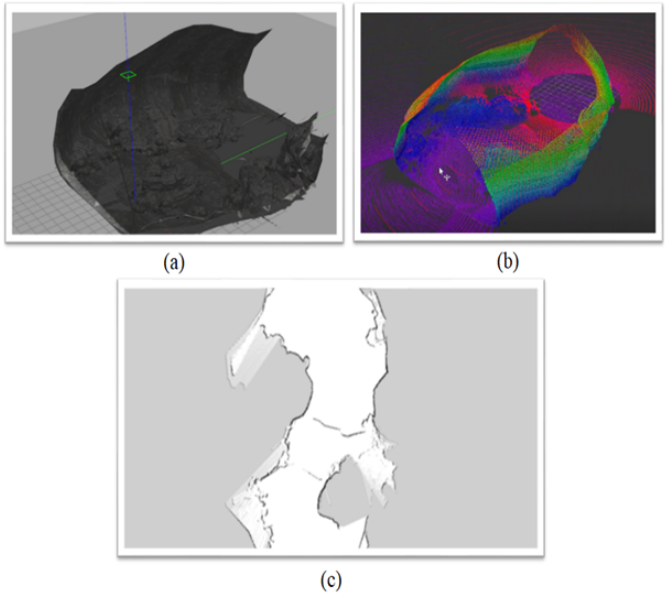


Fig. 8. (a) The simulated lava cave, (b) 3D OctoMap, (c) 2D Gmap

B. Navigation Schemes

1) Navigation using the A Algorithm:* In the case of the A* algorithm, the planned path can be observed from Fig.9 (a). Here the blue line represents the planned path by the algorithm and the grey layer around obstacles was the inflation layer, which the vehicle footprint was to avoid.

2) Navigation using the Dijkstra Algorithm: The global path planned by implementing the Dijkstra algorithm is depicted in Fig. 9 (b) represented by the similar blue line . The following differences were observed which will be discussed in detail in the discussion sub-section.

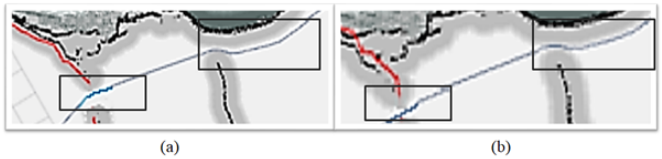


Fig. 9. Highlighted differences between paths planned by A*(a) and Dijkstra (b) algorithms

TABLE II
COMPARATIVE ANALYSIS BETWEEN 3D OCTOMAP AND 2D GMAP

Parameters	Mapping	
	3D OctoMap	2D Gmap
Data Type	Pointcloud	laser scan
Speed	Slower	Faster
Details	3D depth image	2D reference co-ordinate
Sensor	3D Lidar + Wheel Odometry	2D Lidar + Wheel Odometry
Sensor Weight	Heavier $(400-500g)$	Lighter $(100-200g)$
Sensor Power	Higher $(14-20W)$	Lower (<5mW)
Navigation	Possible	Possible

3) Navigation using the RRT Algorithm: Navigation utilizing the RRT algorithm took a different approach where the paths were planned by local and global frontiers. In Fig. 10, the yellow squares represent the local frontier, the red one represents the global frontier and the paths explored are represented by the blue lines.

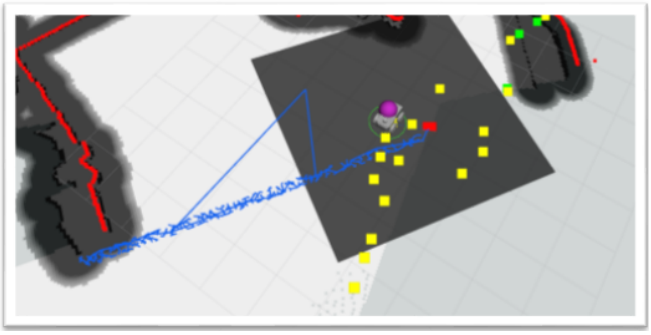


Fig. 10. Navigation using the RRT Algorithm

C. Discussion

1) Result Discussion: It is evident from Fig. 8 (a), (b), and (c), and Table 1 that the 2D Gmap generated by the RRT algorithm possesses noticeable downgrades in comparison with manual Gmapping due to the nature of the path planner.

Again it can be stated according to Table 2 that although 3D mapping is a substantial improvement in scanned environment details, the weight and power consumption for the affiliated sensor integration is also substantially higher in the case of 3D mapping.

Thus, if navigation using both 3D and 2D maps is possible without noticeable performance drop, the latter can be considered as the optimal option if there are significant concerns regarding minimization of vehicle weight and power consumption.

Next, in the case of navigation schemes, despite the fact that A* and Dijkstra provided nearly identical paths, it can be seen in Fig. 9 (a) and (b) that Dijkstra generated shorter but rougher paths in the highlighted regions, where the path proximity to the inflation layer was closer than that of A*. As a result, there was a slight increase in the likelihood of collision and, thus, an increase in time consumption.

TABLE III
COMPARATIVE ANALYSIS AMONG NAVIGATION SCHEMES

Parameters	Navigation Schemes		
	A*	Dijkstra	RRT
Type	Global Path Planner	Global Path Planner	Local Path Planner
Speed	Fastest	Fast	Slowest
Manual Intervention	Was not required	Was not required	Was required
Process Effort	Simpler	Simpler	Complex
Navigation	Possible	Possible	Possible but human intervention is required for complex environments
Obstacle Avoidance	No	No	Yes

Again, in the case of the RRT algorithm, the entire process from mapping to navigation was intended to be autonomous. In a complex environment like ours, however, the robot struggled to generate effective path planning and frequently wandered in the same places. As a result, human intervention was frequently required to manually drive the robot closer to the goal region, and the time consumption was also significant.

Therefore, out of the three navigation schemes, the most optimal results were obtained by using the A* algorithm.

2) Limitations: The most significant limitation of this study is that it was conducted in a simulated environment with mostly ideal conditions, which may differ from the real Martian environment. Again, due to the simulated low gravity, the wheel traction was also reduced which could have resulted in minor mishaps with the wheel odometry data used to generate the map.

V. CONCLUSION

In this paper, we tried to address the performance benchmarks of various navigation schemes using A*, Dijkstra, and RRT algorithms and the trade-off between prominent 2D and 3D slam methods intended to explore a Martian lava cave. The experiments required for the study were carried out in simulated environments using ROS packages and Gazebo simulator to best match the real environment. Gmapping was used as the default mapping process and AMCL was utilized for robot localization. Finally, navigation took place utilizing all of the aforementioned algorithms and a corresponding assessment was performed. From the study, it was evident that A* slightly outperformed Dijkstra in terms of generating optimal path planning, whereas RRT struggled significantly in the complex environment. It was also determined that if there is no significant performance loss when using 2D mapping instead of 3D, the 2D method is optimal because it reduces vehicle weight and power consumption. for future work, we wish to evaluate more advanced local planners on this explorer and implement autonomous exploration on a much complex environment.

REFERENCES

[1] T. Pipan and D. C. Culver, "Shallow subterranean habitats," in *Encyclopedia of caves*. Elsevier, 2019, pp. 896–908.

[2] J. Balaram, M. Aung, and M. P. Golombek, "The ingenuity helicopter on the perseverance rover," *Space Science Reviews*, vol. 217, no. 4, pp. 1–11, 2021.

[3] M. A. Batalin, G. S. Sukhatme, and M. Hattig, "Mobile robot navigation using a sensor network," in *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, vol. 1. IEEE, 2004, pp. 636–641.

[4] M. N. Ab Wahab, C. M. Lee, M. F. Akbar, and F. H. Hassan, "Path planning for mobile robot navigation in unknown indoor environments using hybrid psofs algorithm," *IEEE Access*, vol. 8, pp. 161 805–161 815, 2020.

[5] G. E. Jan, K. Y. Chang, and I. Parberry, "Optimal path planning for mobile robot navigation," *IEEE/ASME Transactions on mechatronics*, vol. 13, no. 4, pp. 451–460, 2008.

[6] A. Niloy, A. Shama, R. K. Chakrabortty, M. J. Ryan, F. R. Badal, Z. Tasneem, M. H. Ahamed, S. I. Moyeen, S. K. Das, M. F. Ali *et al.*, "Critical design and control issues of indoor autonomous mobile robots: A review," *IEEE Access*, 2021.

[7] T. T. Mac, C. Copot, D. T. Tran, and R. De Keyser, "Heuristic approaches in robot path planning: A survey," *Robotics and Autonomous Systems*, vol. 86, pp. 13–28, 2016.

[8] M. Noto and H. Sato, "A method for the shortest path search by extended dijkstra algorithm," in *Smc 2000 conference proceedings. 2000 ieee international conference on systems, man and cybernetics.'cybernetics evolving to systems, humans, organizations, and their complex interactions'(cat. no. 0*, vol. 3. IEEE, 2000, pp. 2316–2320.

[9] J.-H. Zhou and H.-Y. Lin, "A self-localization and path planning technique for mobile robot navigation," in *2011 9th World Congress on Intelligent Control and Automation*. IEEE, 2011, pp. 694–699.

[10] C. Zammit and E.-J. Van Kampen, "Comparison between a* and rrt algorithms for uav path planning," in *2018 AIAA guidance, navigation, and control conference*, 2018, p. 1846.

[11] A. Mandow, J. L. Martinez, J. Morales, J. L. Blanco, A. Garcia-Cerezo, and J. Gonzalez, "Experimental kinematics for wheeled skid-steer mobile robots," in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2007, pp. 1222–1227.

[12] S. Karamchandani, S. Pednekar, A. Pusalkar, S. Bhattacharjee, and D. Issrani, "Autonomous parking system perception and control simulations on ros-gazebo," in *Proceedings of International Conference on Wireless Communication: ICWiCom 2021*, vol. 92. Springer, 2022, p. 345.

[13] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.

[14] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, no. 3, pp. 189–206, 2013.

[15] K. Wei and B. Ren, "A method on dynamic path planning for robotic manipulator autonomous obstacle avoidance based on an improved rrt algorithm," *Sensors*, vol. 18, no. 2, p. 571, 2018.

[16] J. Kuffner and S. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, vol. 2, 2000, pp. 995–1001 vol.2.

[17] X. Li, X. Hu, Z. Wang, and Z. Du, "Path planning based on combinaion of improved a-star algorithm and dwa algorithm," in *2020 2nd International Conference on Artificial Intelligence and Advanced Manufacture (AIAM)*. IEEE, 2020, pp. 99–103.